

NOIP2024 信心赛题解

Tx_Lcy

你 (a)

知识点：【5】 集合 (set)、多重集合 (multiset)

签到题，大家应该都会。

[原题链接](#)

考虑从右往左扫描 r_1 ，相当于枚举了左右区间的分界线。

在扫描的过程中不断把分界线右边的区间加入一个 **set**，然后枚举 l_1 ，查询前驱后继即可。

时间复杂度 $\Theta(n^2 \log n)$ 。

觉得 (b)

知识点：【3】 贪心法

[原题链接](#)

考虑把位置分成三类：左右都比它小/左右都比它大/左右恰好一个比它大。如果位置在边界上，我们根据跟它相邻的数的大小把这个位置分到第一类或第二类里。

这样我们就可以把绝对值拆开了，容易发现第三类位置对答案的贡献系数为 0，其余两类分别为 2 和 -2 。

一个自然的想法是给第一类位置填尽可能大的数，第二类位置填尽可能小的数。

我们发现如果给第一类位置填最大的一些数给第二类位置填最小的一些数是合法的，并且构造方案非常简单：在每个由第一类和第二类位置分割出的连续段中按递增或递减顺序填入剩余的数即可。

于是做完了，时间复杂度 $\Theta(n \log n)$ 。

你很 (c)

知识点：【4】 动态规划的基本思路 【8】 动态规划的常用优化

[原题链接](#)

首先考虑 $T = 1$ 的情况，不难发现此时我们无需考虑“配对是极大的”这个条件，因为不是极大的一定不优。

因此不难设 $dp_{i,j}$ 表示第一种奶牛匹配到第 i 个，第二种奶牛匹配到第 j 个时的答案，前缀 $\max$ 优化转移即可，时间复杂度 $\Theta(n^2)$ 。

考虑 $T = 2$ 怎么做，首先解决如何判断一个方案是否合法。

容易发现匹配不会交叉，不然可以调整为不交叉。因此可以将方案之外的奶牛按照坐标排序，然后如果相邻奶牛都配对不了方案就合法。

因此在原来的基础上加一维表示上一个失配奶牛即可做到 $\Theta(n^3)$ 。

发现现在状态数已经 $\Theta(n^3)$ 了，考虑优化状态数。

不难发现如果失配奶牛种类不变一定合法，所以设 $dp_{i,j,0/1}$ 表示第一种奶牛匹配到第 i 个，第二种奶牛匹配到第 j 个，上一个失配奶牛是第一种/第二种时的答案。

对于失配奶牛不变的转移是容易的，现在我们只需关注 $dp_{i,j,0} \rightarrow dp_{x,y,1}$ 和 $dp_{i,j,1} \rightarrow dp_{x,y,0}$ 的转移即可，我们可以假装此时最后一个失配奶牛是 i/j ，这样显然不会把答案算错。

两种转移本质相同，我们只考虑第一种。

不难发现 $x - i = y - j - 1$ ，并且 $i + 1 \sim x$ 和 $j + 1 \sim y - 1$ 必须能完全匹配，并且奶牛 y 的位置必须在奶牛 i 之后，不然就不是“最后一个失配的奶牛”了。

现在可以轻松做到 $\Theta(n^3)$ ，考虑优化。

设 $\Delta = x - i$ ，不难发现合法的 Δ 一定是一个区间，因此找出这个区间，然后相当于在斜线上做一个前缀 max，做完了，时间复杂度 $\Theta(n^2)$ 。

牛吗 (d)

知识点：【6】 线段树

[原题链接](#)

离线做法

设 f_i 表示右端点为 i 时最右的合法左端点。

考虑删除操作，发现此时可以轻松维护 f_i 的变化，具体地，设删除位置为 x ，我们找出上一个和下一个与 a_x 相同的位置 pl, pr ，相当于令 $j \in [i, pr)$ 的 f_j 对 pl 取 $\min$ ，由于 f 数组单调，容易转化成区间覆盖。

但是加入操作比较难做，所以考虑不加入了。

先加入所有点，然后线段树分治维护删除，容易做到 $\Theta(n \log^2 n)$ 。

但是这个东西比较难写，并不推荐。

在线做法

同样设 f_i 表示右端点为 i 时最右的合法左端点。

考虑 f_i 有什么性质，设 $next_i$ 表示 i 之后第一个等于 a_i 的位置，不难发现 f_i 是最小的 p 满足 $next_p > i$ 。

这个描述仍然有点抽象，我们设 pre_i 表示 i 前面第一个等于 a_i 的位置，发现 $f_i = \min_{j \in [i+1, n]} pre_j$ ，但是如果 i 右边出现的数没有覆盖 $[1, k]$ 就爆了，因此我们令 $a_{n+1}, \dots, a_{n+k} = \{1, \dots, k\}$ 。

不难发现一次单点修改只会产生 $\Theta(1)$ 个 pre_i 的修改，所以可以使用单侧递归线段树轻松维护 f 数组。

注意实际实现时并不需要令 $a_{n+1, \dots, n+k} = \{1, \dots, k\}$ ，只需维护 $[1, k]$ 最后出现的位置即可，时间复杂度 $\Theta(n \log^2 n)$ 。